WELCOME

Into the Box 2025: The Future is Dynamic!

www.intothebox.org

CommandBox WebSockets and SocketBox

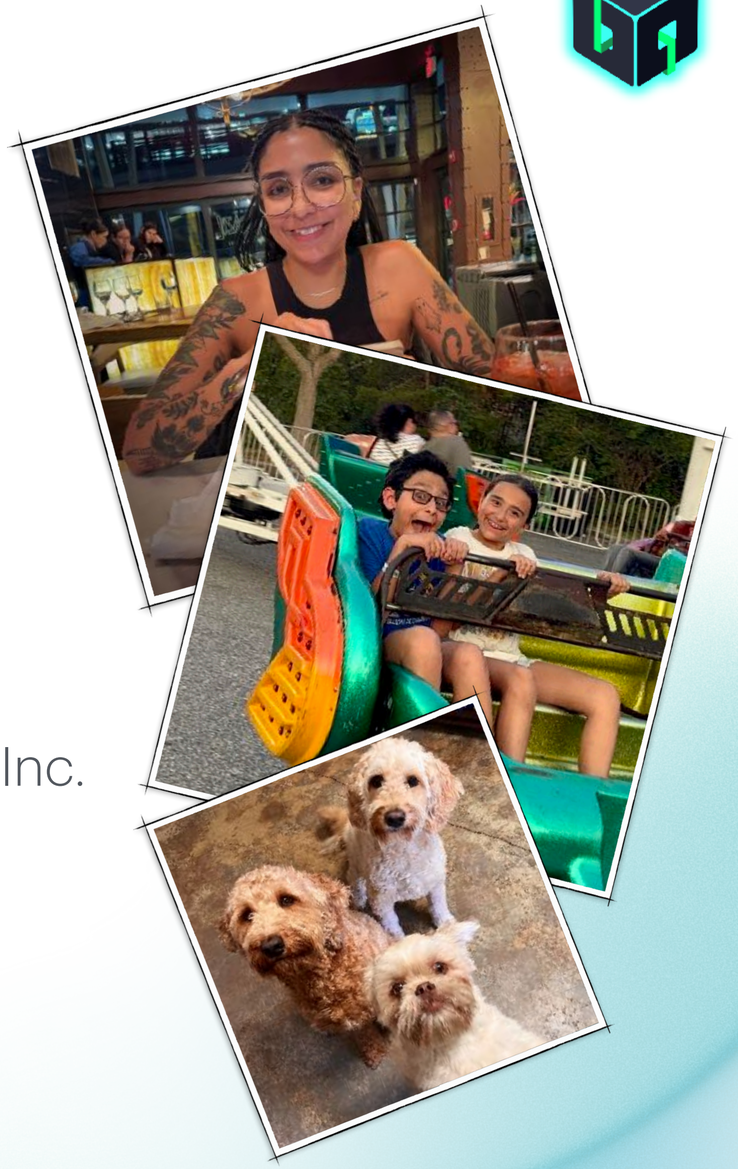
Powerful, Customizable, and Feature rich

Giancarlo Gomez
Into The Box 2025

About Me

Giancarlo Gomez

- Proud Father of 3
- A Dog Person
- Musician
- Web Developer since 1999
- Owner of Fuse Developments, Inc. and CrossTrackr, Inc.
- South Florida ColdFusion User Group Co-Manager



Fasten your Seatbelts !

Let's Kick Into High Gear!
We have a lot to cover ...

What WebSockets Are

- **Persistent Connection**

  Provide a continuous, full-duplex connection between a client and a server, allowing real-time, bidirectional communication without the need to constantly reopen new connections.

- **Stateful Lightweight Protocol**

  Unlike HTTP, they use a minimal overhead protocol once the initial handshake is complete, which reduces bandwidth usage and improves performance and connections stay alive between client and server until it gets terminated by either party.

- **Low Latency**

  Since connections remain open, data can be sent and received almost instantly.

- **Event Driven Web Programming**

  Perfect for applications that require real-time updates, such as live notifications, chat apps, or interactive dashboards, where actions trigger immediate server responses.

What WebSockets Are Not

Push Notifications

Messages sent to iOS or Android apps using Apple Push Notification Service (APNs) and Google Cloud Messaging (GCM)

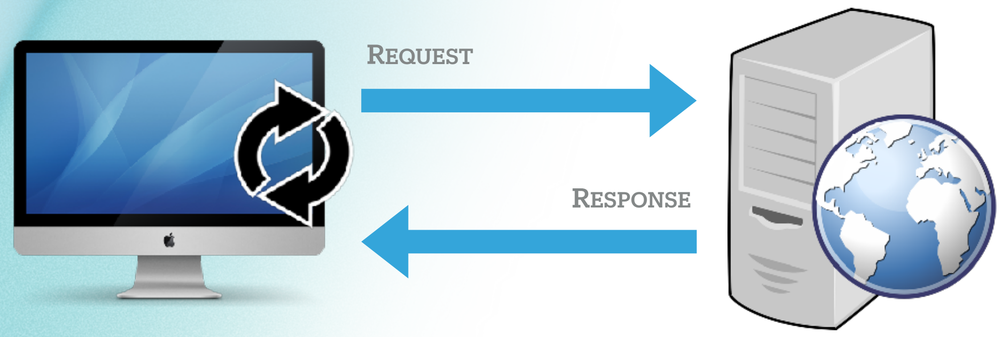
Web Push Notifications

Messages sent from a Website via a Push Service

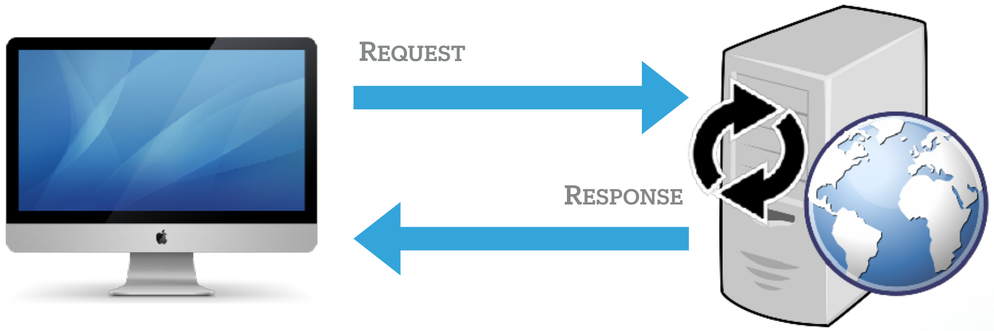
	WebSockets	Push Notifications
Deliverable to closed application	NO	YES
Latency	Realtime	Usually near-realtime
Frequency (throughput)	High	Low

I Already Do This

Short Polling

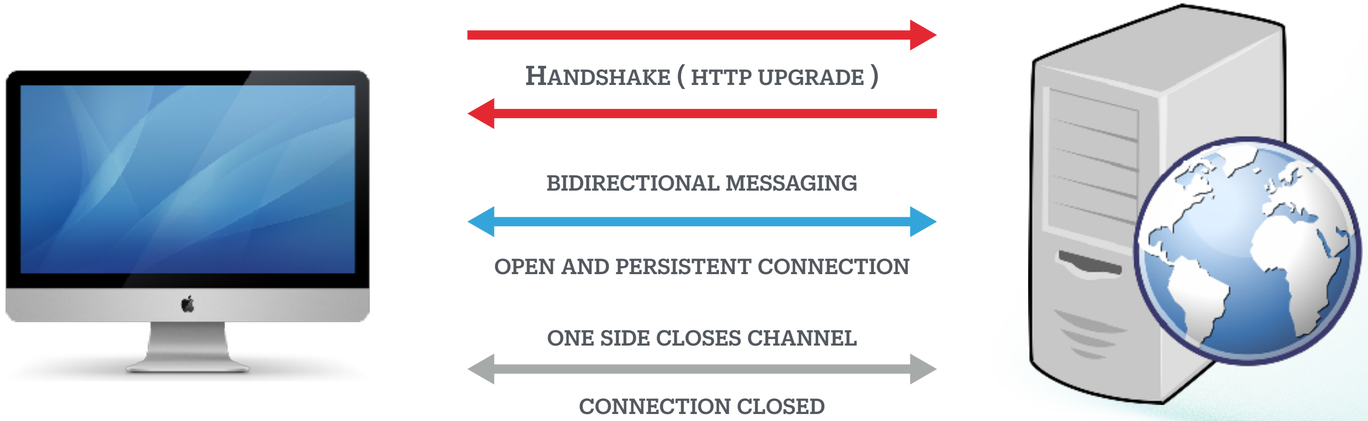


Long Polling



But there is a better way ...

WebSockets



Let's See It In Action!

https://jc-go.link/WebSockets-In-Action

WebSockets In CommandBox

- **Natively Supported via the SocketBox Library.**

  WebSocket Listener library used with CommandBox WebSocket and BoxLang WebSocket server.

  ℹ The WebSocket server is provided by Undertow.

- **Not Vendor / Engine Specific**

  Works with Adobe, Lucee, and BoxLang! Use it in your current applications today!

- **Not a separate server**

  It is an upgrade listener which will upgrade any WebSocket request and pass incoming messages to your application via an internal http request to your listener. This gives you access to everything you are use to, like your session scope, not possible with other solutions!!!

- **Two modes to choose from**

  Core, offering simple WebSocket support, and STOMP broker, with additional features.

- **Easy to configure**

How Do I Use?

- CommandBox 6.1.0 or BoxLang Mini Server
- Install SocketBox

  ```
  box install socketbox
  ```

- Configure in your `server.json`

  Only available for CommandBox as the BoxLang Mini Server uses the defaults and is not configurable.

  ```
  web.websocket.enable     = true|false
  web.websocket.uri        = /ws
  web.websocket.listener   = /WebSocket.cfc
  ```

- Create a custom CFC or BX Class that extends the mode you want to use

  `modules.socketbox.models.WebSocketCore` or `modules.socketbox.models.WebSocketSTOMP`

- Server Side, use the the public methods from your WebSocket class

  ```
  CORE  - sendMessage( message, channel )
  CORE  - broadcastMessage( message )
  STOMP - send( destination, messageData, headers={} )
  ```

- Client Side, write some `JavaScript` and get creative

CommandBox Config

Configure your server easily via `server.json` settings.

- `web.websocket.enable`

  Whether or not the WebSocket handler is active. `true | false`

- `web.websocket.uri`

  The URI the WebSockets listener will listen on, defaults to `/ws`.

- `web.websocket.listener`

  The publicly accessible remote class (cfc or bx) to respond to incoming WebSocket messages. Default is `/WebSocket.cfc` in the web root.

ℹ In multi-site configurations, you can set at the root level or at the server individual level via `sites.sitename.websocket.{setting}`

```
{
    "name":"WebSocket.MultiSite.Demo",
    "app":{
        "cfengine":"boxlang"
    },
    "web":{
        "websocket":{
            "enable":true
        }
    },
    "sites":{
        "site1":{
            "hostAlias":"site1.example.com",
            "webroot":"site1/",
            "websocket":{
                "enable":false
            }
        },
        "site2":{
            "hostAlias":"site2.example.com",
            "webroot":"site2/",
            "websocket":{
                "listener":"services/WebSocket.cfc"
            }
        }
    }
}
```

WebSocket Core

Has:

- Low level connection management
- Incoming message handling
- Send messages to one or all connected clients

Does not have:

- Authentication
- Authorization
- Message Routing
- Conventions of what is inside of a message

```
component extends="modules.socketbox.models.WebSocketCore" {

    // called for every new remote connection
    function onConnect( required channel ) { }

    // called every time a connection is closed
    function onClose( required channel ) { }

    // channel here represents the connected client
    function onMessage( required message, required channel ) {
        // send a message back to a specific client
        // in this case, we are sending a message back
        // to the client that sent the message
        if ( arguments.message == "Ping" ) {
            sendMessage( "Pong", arguments.channel );
        }
        // send the message to all connected clients
        else {
            broadcastMessage( arguments.message );
        }
    }

}
```

You can build whatever you want on top of this basic functionality!

WS Core Methods

Methods you can override

- `onConnect( channel )`

  Called on every new connection.

  💡 You can add logic to secure your server and track the connections.

- `onClose( channel )`

  Called each time a connection is closed.

  💡 If you are tracking your connections, this is where you would add logic to remove it.

- `onMessage( message, channel )`

  Called every time an incoming message is received.

  ⚠ At minimum, you have to define this, if not, messages will not be delivered.

Methods inherited from `WebSocketCore` you can use

These are the methods that you can use inside your WebSocket CFC / Class and within your application.

- `sendMessage( message, channel )`

  Send a message to a specific client

- `broadcastMessage( message )`

  Send a message to all connected clients

- `getAllConnections()`

  Returns an array of all channels representing every remote connection to the server

```
component extends="modules.socketbox.models.WebSocketCore" {

    function onConnect( required channel ) {
        // do not allow the connection if the user is not logged in
        if ( !session.keyExists( "user" ) ){
            channel.close();
        }
        else {
            // add the connection to our subscribers struct
            getSubscribers()[ arguments.channel.hashCode() ] = arguments.channel;
            // send connection success message to the client with their id
            sendMessage(
                serializeJSON( { event:"connected", id:channel.hashCode() } ),
                arguments.channel
            );
        }
    }

    function onClose( required channel ) {
        // remove the connection from our subscribers struct
        getSubscribers().delete( arguments.channel.hashCode() );
    }

    function onMessage( required message, required channel ) {
        // always send the message as a JSON object
        if ( arguments.message == "Ping" ) {
            sendMessage(
                serializeJSON( { event:"pong" } ),
                arguments.channel
             );
        }
        else {
            broadcastMessage(
                serializeJSON( {
                    event       : "message",
                    data        : isJSON( arguments.message ) ?
                                    deserializeJSON( arguments.message ) :
                                    arguments.message,
                    publisherID : channel.hashCode()
                } )
            );
        }
    }

    function getSubscribers() {
        setup();
        return application.socketBoxSubscribers;
    }

    private function setup(){
        cflock( name="WebSocketAdvancedSetup", type="exclusive", timeout=10 ) {
            if ( !application.keyExists( "socketBoxSubscribers") )
                application.socketBoxSubscribers = {};
        }
    }

}
```

WS Core Client

Connecting from the Client

Simply use the WebSocket API to connect 🚀

https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

```
const ws = new WebSocket(
    `${location.protocol.indexOf('https') == -1 ? 'wss' : 'ws'}` +
    `://${location.host}/ws`
);

ws.onopen = function() {
    console.log( 'Connected to WebSocket server' );
};

ws.onmessage = event => {
    let data = event.data;
    try { data = JSON.parse( data ); }
    catch (e) {}
    console.log( data );
};

ws.onclose = event => {
    console.log( 'Socket is closed.', event );
};

ws.onerror = error => {
    console.error( 'Socket encountered error: ', error.message );
};
```

STOMP Broker

STOMP (Simple Text Oriented Messaging Protocol)

- Authentication

  Accept or deny connect requests.

- Authorizing

  Allow or deny clients from subscribing or publishing to a destination.

- Subscriptions to Destinations

  Referred to as channels or rooms in other implementations.

- Message Format / Serialization
- Message Routing
- Automatic Reconnect
- Heartbeat Messages

```
component extends="modules.socketbox.models.WebSocketSTOMP" {

    // configures the STOMP Broker
    function configure() {
        return { };
    }

    // custom authentication logic - can I connect?
    boolean function authenticate( ... ) {
        return true;
    }

    // custom authorization logic - can I send a message?
    boolean function authorize( ... ) {
        return true;
    }
}
```

SB Configuration

- debugMode

  Reloads config on every request and additional logging to the console.

- heartBeatMS

  Controls the number of ms between heartbeat "pings" from every client WebSocket connection. Set to 0 to disable.

- Exchanges

  - direct

    Routes message directly to the destination matching their destination header.

  - topic

    Routes messages to destinations via wildcard subscriptions allowing partial match semantics on destination names.

  - fanout

    Routes message to ALL bindings at the same time.

  - distribution

    Routes messages to a SINGLE outgoing destination based on a selection algorithm.

ℹ Destinations can be created in real-time by subscribers. Meaning there is no need to configure any destinations. The exchanges simply assist in routing.

```
component extends="modules.socketbox.models.WebSocketSTOMP" {

    function configure() {
        return {
            // Reload the config on EVERY request (for development only!)
            debugMode : true,
            // How often to send a heartbeat to the client (in milliseconds)
            heartBeatMS : 10000,
            // Configure the exchanges and their bindings which will process incoming messages.
            exchanges : {
                // Direct exchange routes messages to a single destination
                direct : {
                    bindings : {
                        "rock" : "rock-music",
                        "rap"  : "rap-music"
                    }
                },
                // Topic exchange routes messages based on a pattern match
                topic : {
                    bindings : {
                        "music.*"    : "all-music",
                        "music.rock" : "rock-music",
                        "music.rap"  : "rap-music"
                    }
                },
                // Fanout exchange routes messages to all bound destinations
                fanout : {
                    bindings : {
                        "all-music" : [ "rock-music", "rap-music" ]
                    }
                },
                // Distribution exchange routes messages to a single destinations
                // at a time in a round-robin or random fashion
                distribution : {
                    type : "roundrobin", // roundrobin or random
                    bindings : {
                        "distribute-music" : [ "rock-music", "rap-music" ]
                    }
                }
            },
            // Define server-side listeners
            subscriptions : {
                "global-music" : ( message ) => {
                    send(
                        "all-music",
                        { "data": "This is for all music!!!" }
                    );
                }
            }
        };
    }

    // custom authentication logic - can I connect?
    boolean function authenticate( ... ) { return true; }

    // custom authorization logic - can I subscribe / publish to a destination?
    boolean function authorize( ... ) { return true; }
}
```

SB Methods

Methods you can override

- `configure()`

  Configure your STOMP Broker with a struct returned from this method.

- `authenticate( login, passcode, host, channel )`

  Custom authentication logic to accept or deny a connection request.

- `authorize( login, exchange, destination, access, channel )`

  Custom authorization logic to allow or deny a client from being able to subscribe and/or publish to a destination.

Methods inherited from `WebSocketSTOMP` you can use

These are the methods that you can use inside your WebSocket CFC / Class and within your application.

- `send( destination, messageData, headers )`

  Send a message to subscribers of a destination (direct or routed via the exchange).

- `getSubscriptions()`

  Get all current STOMP subscriptions, which includes browser clients and server side listeners.

- `getConnectionDetails( channel )`

  Get details for a given channel.

```
component extends="modules.socketbox.models.WebSocketSTOMP" {

    function configure() {
        return { ... };
    }

    /**
     * Authenticate the client when they connect to the server
     *
     * @login    The username or token provided by the client
     * @passcode The password or token provided by the client
     * @host     The host the client is connecting from
     * @channel  The channel object for the connection
     */
    boolean function authenticate(
        required string login,
        required string passcode,
        string host,
        required channel
    ) {

        // Example : Add your own authentication logic here
        if ( arguments.login != "admin" || arguments.passcode != "password" )
            return false;

        return true;
    }

    /**
     * Authorize the client when they try to subscribe or publish to a destination
     *
     * @login       The username or token provided by the client when they authenticated
     * @exchange    The exchange the client is trying to subscribe or publish to
     * @destination The destination the client is trying to subscribe or publish to
     * @access      The type of access the client is trying to get "read" | "write"
     * @channel     The channel object for the connection
     */
    boolean function authorize(
        required string login,
        required string exchange,
        required string destination,
        required string access,
        required channel
    ) {
        var connectionDetails = getConnectionDetails( arguments.channel );
        var sessionID         = connectionDetails[ "sessionID" ] ?: "";

        // Example : Only let people subscribe to their own ping/pongs
        if (
            arguments.destination.lcase().startsWith( "pong." ) &&
            arguments.destination != "pong." & sessionID
        )
            return false;

        return true;
    }
}
```

SB Message Object

Messages are composed of 3 parts

- `command` - Purpose of the message
- `headers` - Struct of metadata describing the message
- `body` - String (JSON if complex object)

Methods available on messages when received by your listener

- `getCommand()`
  most likely always be "message"
- `getBody()`
  Will auto-deserialize JSON back into complex objects when the content-type header is application/json
- `getHeaders()`
  Struct of headers
- `getHeader( key, defaultValue )`
  Get a single header, with an optional default value
- `getBodyRaw()`
  Struct of headers
- `getChannel()`
  The channel which sent the original message (null if sent server - side)

```
MESSAGE
content-length:41
destination:global
subscription:sub-0
message-id:E483E88A-7F6C-4E4A-B685F70526234850
publisher-id:829137625

{"notify":"Client 829137625 has arrived"}NUL
```

```
"subscriptions" : {
    "jokes" : ( message ) => {
        systemOutput( "THE COMMAND : " & message.getCommand() );
        systemOutput( message.getBody() );
        systemOutput( message.getBodyRaw() );
        systemOutput( message.getHeaders() );
        systemOutput(
            isNull( message.getChannel() ) ?
            "SERVER" :
            message.getChannel().hashCode()
        );
    }
}
```

SB Client

The STOMP broker should be compatible with any STOMP client.

- **Recommended Library**
  https://www.npmjs.com/package/@stomp/stompjs
- **Getting Started Guide**
  https://stomp-js.github.io/guide/stompjs/using-stompjs-v5.html
- **API Docs**
  https://stomp-js.github.io/api-docs/latest/classes/Client.html

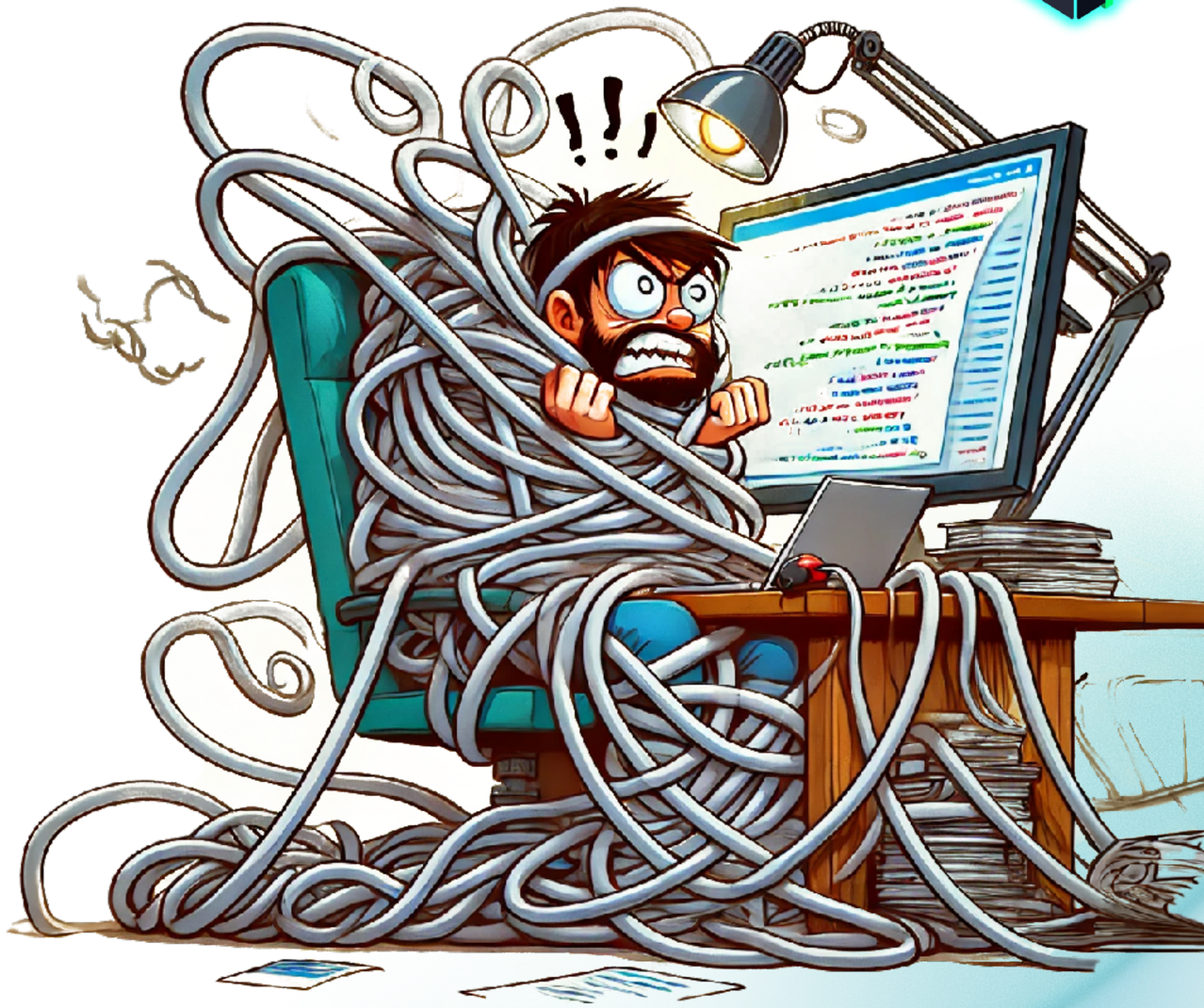
The example is based on ESM, but you can also opt for a UDM version which makes the Client library available globally @ StompJs.Client.

```
<script src="https://cdn.jsdelivr.net/npm/@stomp/stompjs@7.1.1/bundles/stomp.umd.min.js"></script>
<script>
    const client = new StompJs.Client( { ... } );
</script>
```

```
<script type="module">
    import { Client } from 'https://cdn.jsdelivr.net/npm/@stomp/stompjs@7.1.1/+esm';

    const client = new Client({
        brokerURL : `${location.protocol.indexOf('https') == -1 ? 'wss' : 'ws'}` +
                    `://${location.host}/ws`,
        reconnectDelay : 5000,
        heartbeatIncoming : 10000,
        heartbeatOutgoing : 10000,
        connectHeaders    : {
            login    : '',
            passcode : '',
            host     : location.hostname
        },
        debug: function ( str ) {
            console.log( str );
        },
        onConnect: function ( frame ) {
            // set the id of the session
            this.id = parseInt( frame.headers.session );
            // use to track my subscriptions and easily unsubscribe
            this.subscriptions = {};

            this.subscriptions.global = this.subscribe( 'global', onMessage );

            this.publish({
                destination : 'global',
                body        : JSON.stringify( { notify: `Client ${this.id} has arrived` })
            })
        },
        onStompError: function ( frame ) {
            console.error( `Broker reported error: ${frame.headers.message}` );
            console.error( `Additional details: ${frame.body}` );
        },
        onWebSocketClose: function ( event ) {
            console.log( 'WebSocket closed', event );
        },
        onWebSocketError: function ( event ) {
            console.error( 'WebSocket error', event );
        },
    });

    const onMessage = ( message ) => {
        let body = message.body;
        try { body = JSON.parse( message.body ); }
        catch (e) {}
        console.log( message.headers, body );
    }

    client.activate();

    // make client available globally for working with it in the console
    globalThis.client = client;
</script>
```

Debugging

We all have to do it ...

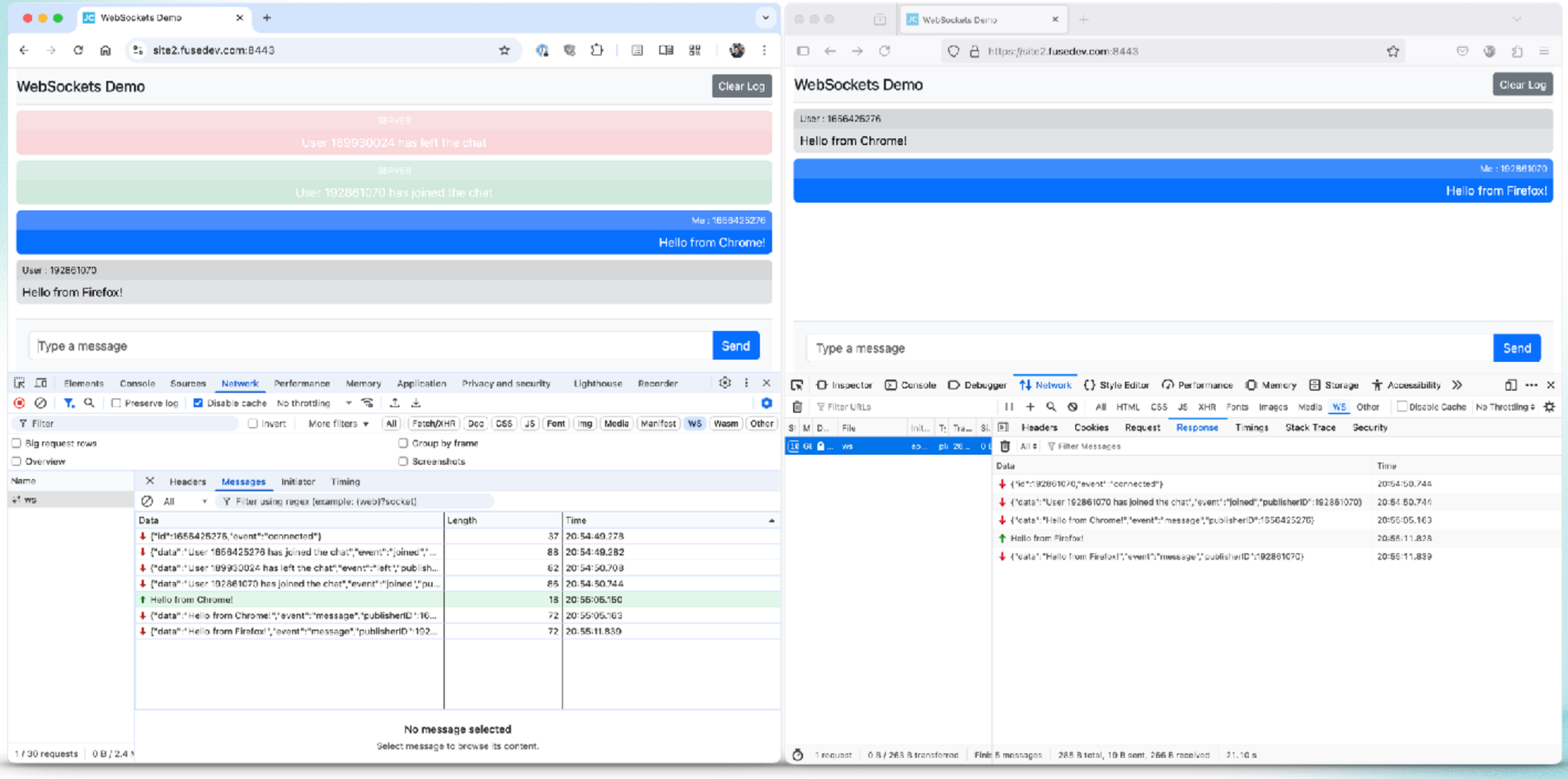


Debugging in Browsers

View messages in your browser's Web Developer Tools

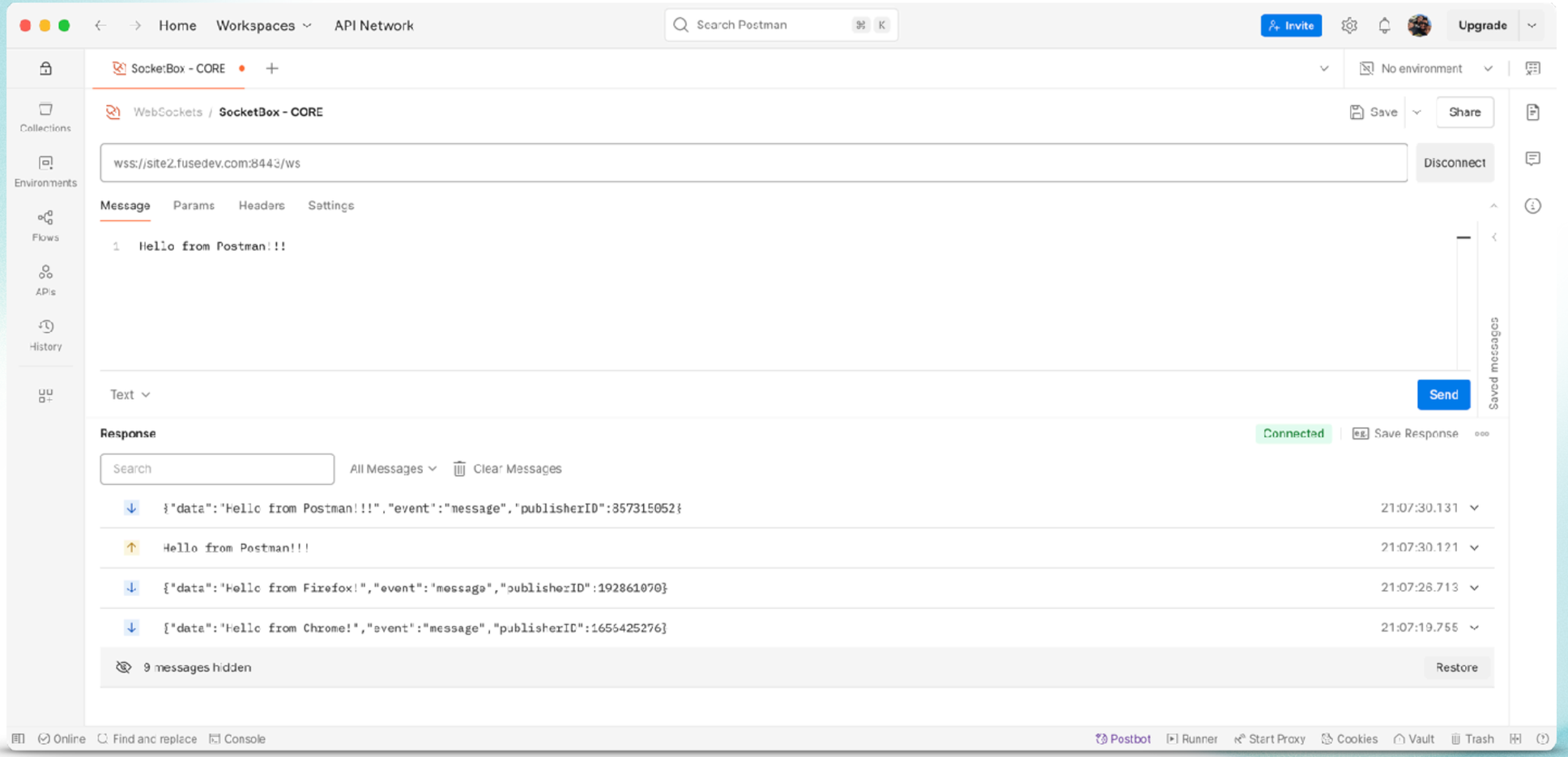
`Web Developer Tools > Network > WS`

Click on entry to inspect, naming convention varies by vendor.



Debugging with Postman

View messages using Postman's WebSocket Request Tools



Live Coding Section!

Presentation Gods Don't Fail Me Now!

No Code was harmed in this demo

How to Contact Me

https://giancarlogomez.dev

giancarlo.gomez@gmail.com

@GiancarloGomez

https://github.com/GiancarloGomez

https://www.linkedin.com/in/giancarlogomez

Q & A

Don't be shy ...